

Pharmacy Management System

OOP Lab Project · Complete Class-by-Class Blueprint

8

Classes

3

Core OOP
Concepts

5

File Operations

2

User Roles

4

Alert Types

3

Report Types

Recommended Build Order

Step 1	Medicine.java	Base class — fields, CRUD, search, array of objects
Step 2	FileManager.java	Save and load inventory from file — persistence
Step 3	Sales.java + Bill.java	Sell medicine, reduce stock, create bill, log sale
Step 4	Admin.java	Password login, restricted access to delete and price changes
Step 5	AlertSystem.java	Expiry and low stock checks triggered on startup
Step 6	Menu.java	Main menu, Scanner input, ties all classes together
Step 7	Extra Features	Discounts, refunds, reports, backup, multiple roles
Step 8	GUI	Add later when taught — Menu class makes this easy to plug in

Class Relationship Overview

```
Menu.java (entry point – controls all user interaction)
|
+---> Admin.java / Staff.java (extends abstract User)
|     password login, role-based access control
|
+---> Medicine.java (inventory array, CRUD, search)
|     PrescriptionMedicine extends Medicine
|
+---> Sales.java (sell medicine, profit/loss)
|     +-- creates --> Bill.java (one bill per transaction)
|     +-- calls  --> FileManager.java (log the sale)
|     +-- updates --> Medicine (reduce stock)
|
```

```

+---> AlertSystem.java          (expiry + low stock – runs on startup)
|
|      +-- reads from --> Medicine inventory
|      +-- logs to    --> FileManager
|
+---> FileManager.java          (ALL file read/write)
      inventory.txt | sales_log.txt | activity_log.txt | backups/

```

Class-by-Class Blueprint

0
1

Medicine

Medicine.java

FIELDS / ATTRIBUTES

- String name
- String company / supplier
- double price
- String expiryDate
- int quantity (current stock)
- int minStockThreshold
- String category
- boolean prescriptionRequired
- int medicineID (unique auto-generated)

METHODS

- addMedicine() — add to inventory array
- deleteMedicine(id) — admin only
- updateMedicine(id) — edit any field
- searchByName(name)
- searchByID(id)
- viewAll() — display full inventory
- getters and setters for all fields

LINKED TO

- FileManager — save inventory changes
- AlertSystem — provides stock/expiry data
- Sales — stock reduced on sale
- Admin — delete and price changes restricted

OOP CONCEPTS USED

- Encapsulation — all fields private
- Inheritance — PrescriptionMedicine extends this
- Array of objects for inventory

0
2

PrescriptionMedicine

PrescriptionMedicine.java

FIELDS / ATTRIBUTES

- Inherits all fields from Medicine
- String prescriptionNote

METHODS

- Inherits all methods from Medicine
- @Override viewAll() — shows prescription warning
- getPrescriptionNote()

LINKED TO

- Medicine — parent class
- Sales — extra warning shown when selling

OOP CONCEPTS USED

- Inheritance — extends Medicine
- Polymorphism — override viewAll()
- Demonstrates IS-A relationship

0
3

Sales

Sales.java

FIELDS / ATTRIBUTES

- ArrayList salesLog (list of Bill objects)
- double totalRevenue
- double totalProfit
- double totalLoss

METHODS

- sellMedicine(id, quantity)
- applyDiscount(percent) — admin only
- processRefund(billID) — reverse a sale
- getDailySummary(date)
- getWeeklySummary() / getMonthlySummary()
- getBestSellingMedicine()
- calculateProfitLoss()

LINKED TO

- Medicine — reduce stock on sale
- Bill — creates one Bill per transaction
- FileManager — log every sale
- Admin — discount and refund restricted

OOP CONCEPTS USED

- Encapsulation — revenue fields private
- Composition — contains Bill objects
- File I/O via FileManager

FIELDS / ATTRIBUTES

- int billID
- String medicineName
- int quantitySold
- double pricePerUnit
- double discountApplied
- double totalAmount
- String dateOfSale
- String soldBy (staff name)

METHODS

- generateBill() — calculates total
- printBill() — formatted receipt
- getBillID()
- getDateOfSale()
- getTotalAmount()

LINKED TO

- Sales — created by Sales on every transaction
- FileManager — bill saved to sales log

OOP CONCEPTS USED

- Encapsulation — all fields private
- Composition — Sales HAS-A Bill
- Single responsibility — only handles billing

0
5

User (abstract) + Admin + Staff

Admin.java

FIELDS / ATTRIBUTES

- String username
- String password (private)
- String role (ADMIN or STAFF)
- int loginAttempts (max 3 before logout)

METHODS

- abstract login() — subclasses implement
- Admin: deleteMedia(id)
- Admin: changePrice(id, newPrice)
- Admin: applyDiscount(percent)
- Admin: viewActivityLog()
- Staff: sellMedicine()
- Staff: viewInventory()
- logActivity(action) — records everything

LINKED TO

- Medicine — admin controls delete + price
- Sales — admin controls discount + refund
- FileManager — activity log saved to file
- Menu — login required before access

OOP CONCEPTS USED

- Abstraction — abstract User class
- Inheritance — Admin and Staff extend User
- Polymorphism — login() behaves differently
- Encapsulation — password is private

0
6

FileManager

FileManager.java

FIELDS / ATTRIBUTES

- String inventoryFilePath
- String salesLogFilePath
- String activityLogFilePath
- String alertLogFilePath
- String backupFolderPath

METHODS

- loadInventory() — reads on startup
- saveInventory() — writes on exit
- logSale(Bill) — appends to sales log
- loadSalesLog() — reads all past sales
- logActivity(action)
- generateBackup() — dated backup copy
- exportDailyReport(date)
- exportWeeklyReport() / exportMonthlyReport()

LINKED TO

- Medicine — load/save inventory
- Sales + Bill — log every transaction
- Admin — save activity log
- AlertSystem — reads inventory for checks

OOP CONCEPTS USED

- Single Responsibility — all file ops here
- File I/O — core of this class
- Exception Handling — FileNotFoundException

0
7

AlertSystem

AlertSystem.java

FIELDS / ATTRIBUTES

- int lowStockThreshold (default)
- int expiryWarningDays (warn X days before)
- ArrayList alertHistory (past alerts)

METHODS

- checkAllOnStartup() — runs all checks
- checkExpiry() — flags expiring soon
- checkLowStock() — flags below threshold
- showAlerts() — print warnings
- saveAlertHistory() — log via FileManager
- getAlertHistory() — view past alerts

LINKED TO

- Medicine — reads stock and expiry data
- FileManager — saves alert history
- Menu — alerts shown before menu appears

OOP CONCEPTS USED

- Encapsulation — thresholds configurable
- Exception Handling — date parsing
- Separation of concerns — alerts isolated

FIELDS / ATTRIBUTES

- Scanner scanner
- Medicine[] inventory (loaded on startup)
- User currentUser (logged in user)
- FileManager fm (shared instance)
- AlertSystem alertSystem

METHODS

- main() — entry point
- showMainMenu() — print options
- showAdminMenu() — admin-only options
- showStaffMenu() — staff options
- handleInput(choice) — routes to correct action
- startup() — load inventory + run alerts
- shutdown() — save inventory + backup

LINKED TO

- ALL classes — Menu is the controller
- Admin/Staff — login before any access
- AlertSystem — run on startup
- FileManager — startup load, shutdown save

OOP CONCEPTS USED

- Polymorphism — currentUser is Admin or Staff
- Composition — contains all class instances
- Separation of UI from logic
- Easy to replace with GUI later

OOP Concepts Coverage

OOP Concept	Where Used	How
Encapsulation	All classes	All fields private, accessed via getters/setters
Inheritance	Medicine → PrescriptionMedicine User → Admin, Staff	Subclasses extend parent, reuse code
Polymorphism	User ref (Admin/Staff) Medicine array	login(), viewAll() behave differently at runtime
Abstraction	User (abstract class)	Admin and Staff forced to implement login()
File I/O	FileManager	All persistence through one dedicated class
Exception Handling	FileManager, Sales	FileNotFoundException, invalid input, out of stock
ArrayList	Sales, AlertSystem	Dynamic list of bills and alerts
Array of Objects	Medicine	Fixed inventory array, objects are Medicine instances

Data Files Structure

File	Format	Contents	Updated When
------	--------	----------	--------------

inventory.txt	CSV per line	All medicine records	Add / update / delete / sell
sales_log.txt	CSV per line	Every bill/transaction	Every sale or refund
activity_log.txt	Plain text	Admin actions + timestamp	Every admin action
alerts_log.txt	Plain text	Past expiry and stock alerts	Every program startup
backup/inventory_DD-MM-YYYY	CSV	Dated backup copy	Every program shutdown

Important Coding Rules to Remember

- One **public class** per file — filename must match the public class name exactly.
- All related helper classes go in the **same file** to avoid ClassNotFoundException.
- All fields should be **private** — always use getters and setters.
- Admin-only methods must **check role** before executing — throw exception if not admin.
- FileManager should be the **only class** that touches files directly.
- AlertSystem runs **before the menu is shown** — always check on startup.
- Every sale must create a **Bill object** and log it via FileManager immediately.
- Use **try-catch** around all file operations — files may not exist on first run.
- Medicine array size should be **fixed and generous** (e.g. 100) since ArrayList may not be taught yet.
- Keep Menu class **only for input/output** — no business logic inside Menu.

Pharmacy Management System — OOP Lab Project Blueprint